

Fundamentals of Software Construction

Chapter 8 Database Programming

School of Computer Science
Xiangyang Jia (贾向阳)



武汉大学
WUHAN UNIVERSITY



Content

- **Database Basics**
- **ADO.NET**
- **ADO Programming Basics**
- **Entity Framework**





Database

➤ What is a Database?

- A collection of related data stored in a certain organizational manner

➤ Database Types

- Relational Databases
 - **MySQL**、Post SQL、Oracle、SQL Server
 - Embedded database: SQLite、Access (For single use desktop application)
- No-SQL Databases
 - In-memory databases: Redis (key-value DB for cache) , MongoDB (Document DB)
 - Columnar database: Hbase、ClickHouse (For large scale big data)
 - Graph databases: Neo4J



Relational Databases

- Organize data into one or more correlated tables
 - Less redundant data, better data integrity
- **DBMS: Database Management Systems**
 - MySQL, Oracle, SQL Server
 - SQLite, Access



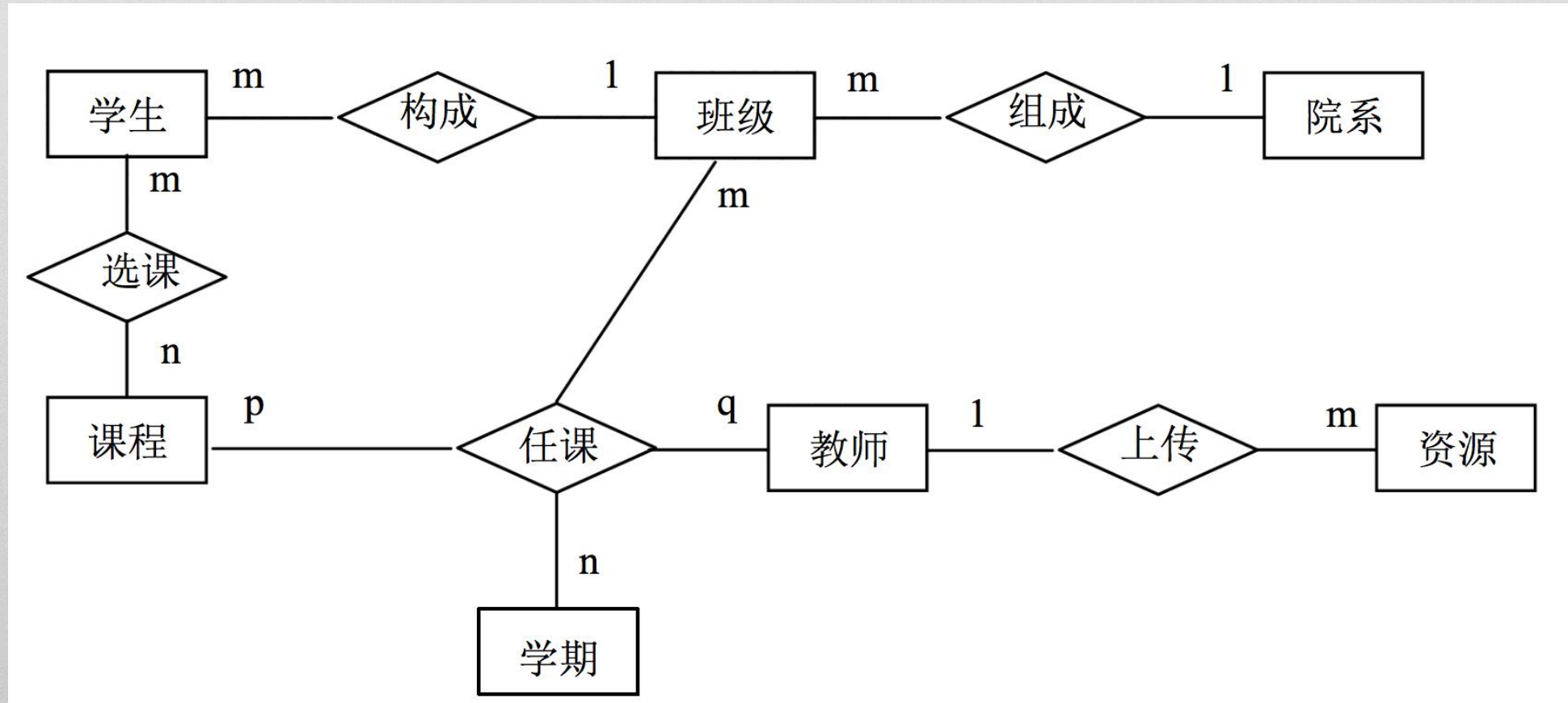
Relational Databases

- **Databases and Tables** : a database contains multiple correlated tables
- **Records**: table rows
- **Fields**: table columns
- **Primary Key**: uniquely identified field of each records
- **Foreign Key**: the field is the primary key of another table



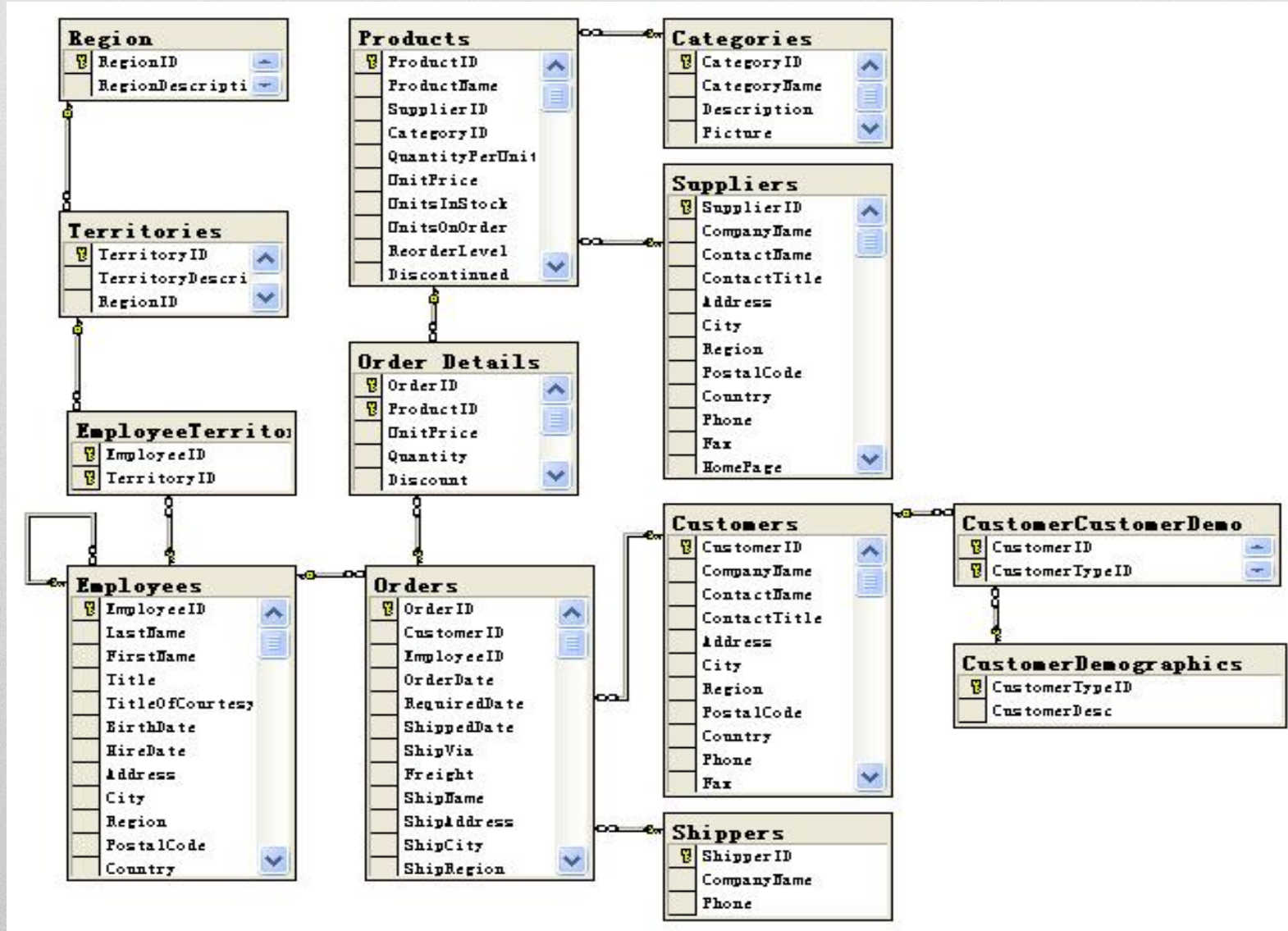
E-R Diagram

- The E-R diagram of a teaching management system





E-R diagram of an order management system





Manage the data in Database

➤ Visual database administration tools

- Navicat, MySQL WorkBench, SQL Server Client, PL/SQL Developer

➤ SQL

- Execution SQL in administration tools
- Programming based on SQL

SQL

- SQL (Structured Query Language)
- The user does not need to tell the database how to access the database, just tell what it needs from the database with SQL.
- The standard language for relational database operations.



➤ Commands

- Database/Table operations: **Create**、 **Drop**
- Query data: **Select**
- Modify data: **Insert**、 **Update**、 **Delete**
- Data Control: **Grant**、 **Commit**、 **Rollback**



SELECT

➤ SELECT- FROM-WHERE

```
SELECT * FROM [publishers]
```

```
SELECT name, age, sex, salary + bonus  
FROM employee  
WHERE depart= '销售部' AND title= '经理' ORDER BY age DESC
```

```
SELECT avg( salary ), max(age), sum(salary)  
FROM employee GROUP BY title
```





SELECT

➤ Key words

- **SELECT** Fields: 包含查询结果要显示的字段清单，字段之间用逗号分开；
- **FROM** Table/View: 限定数据来源，即显示字段的内容来自一个或多个表；
- **WHERE** Condition: 说明查询条件，即选择记录的条件；
- **GROUP BY** Fields: 用于对查询进行分组，可利用它进行分组汇总；
- **HAVING** Condition: 用来限定分组必须满足的条件，必须与GROUP BY连用；
- **ORDER BY** Fields[ASC | DESC] : 用来对查询的结果进行排序，ASC—升序，DESC—降序。



INSERT

- Insert a new record into the table

```
INSERT INTO <Table> [<Field1> [, <Field2>] ...]  
VALUES (<Value1> [, <Value2>] ...)
```

- E.g., Insert a record of a book into the table 'Book'

```
INSERT INTO Book VALUES ('29880004', 'VB.NET基础教程'  
, '张鑫', '清华大学出版社', '2009-9-20', 10, 30)
```




DELETE

- Delete records satisfying the condition from a table

```
DELETE FROM <Table> WHERE <Condition>
```

- Note: Once the records are deleted with a delete query, they cannot be recovered.
- E.g., delete all books whose price is lower than 10:

```
DELETE FROM Book WHERE price<10.0
```




UPDATE

- Update records satisfied the condition

```
UPDATE <Table>  
SET <Field1>=<Exp1>[,<Field2>=<Exp2>]...  
WHERE <Condition>
```

- E.g., Reduce the price of books between \$30 and \$50 to 80% of the original price:

```
UPDATE Book  
SET price=price*0.8  
WHERE price>30 AND price<50
```


Content

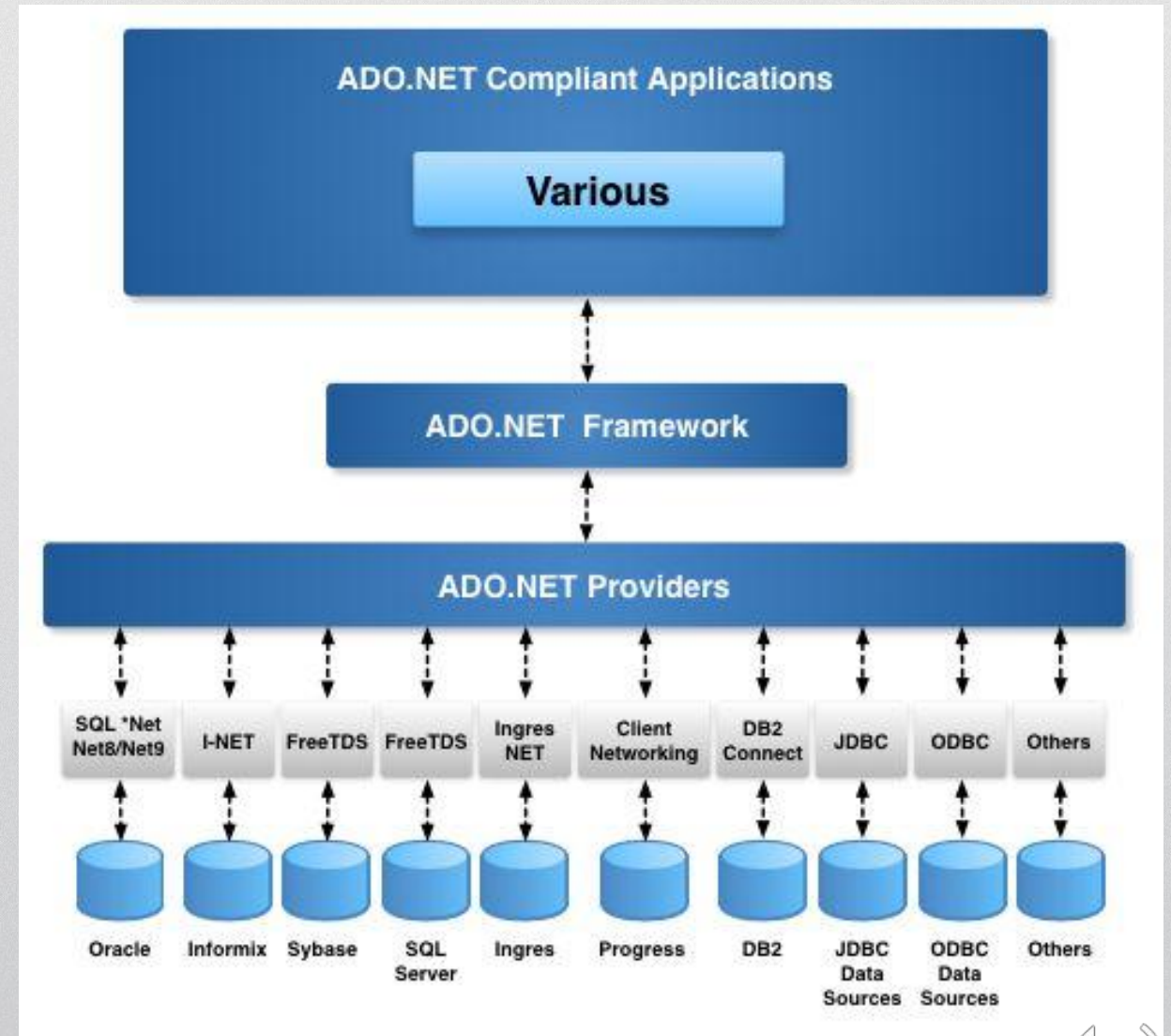
- Database Basics
- **ADO.NET**
- ADO Programming Basics
- Entity Framework





ADO.NET

- **ADO.NET**
 - Namespace: System.Data
 - Provides standard interface for adding, deleting, and querying data from different databases

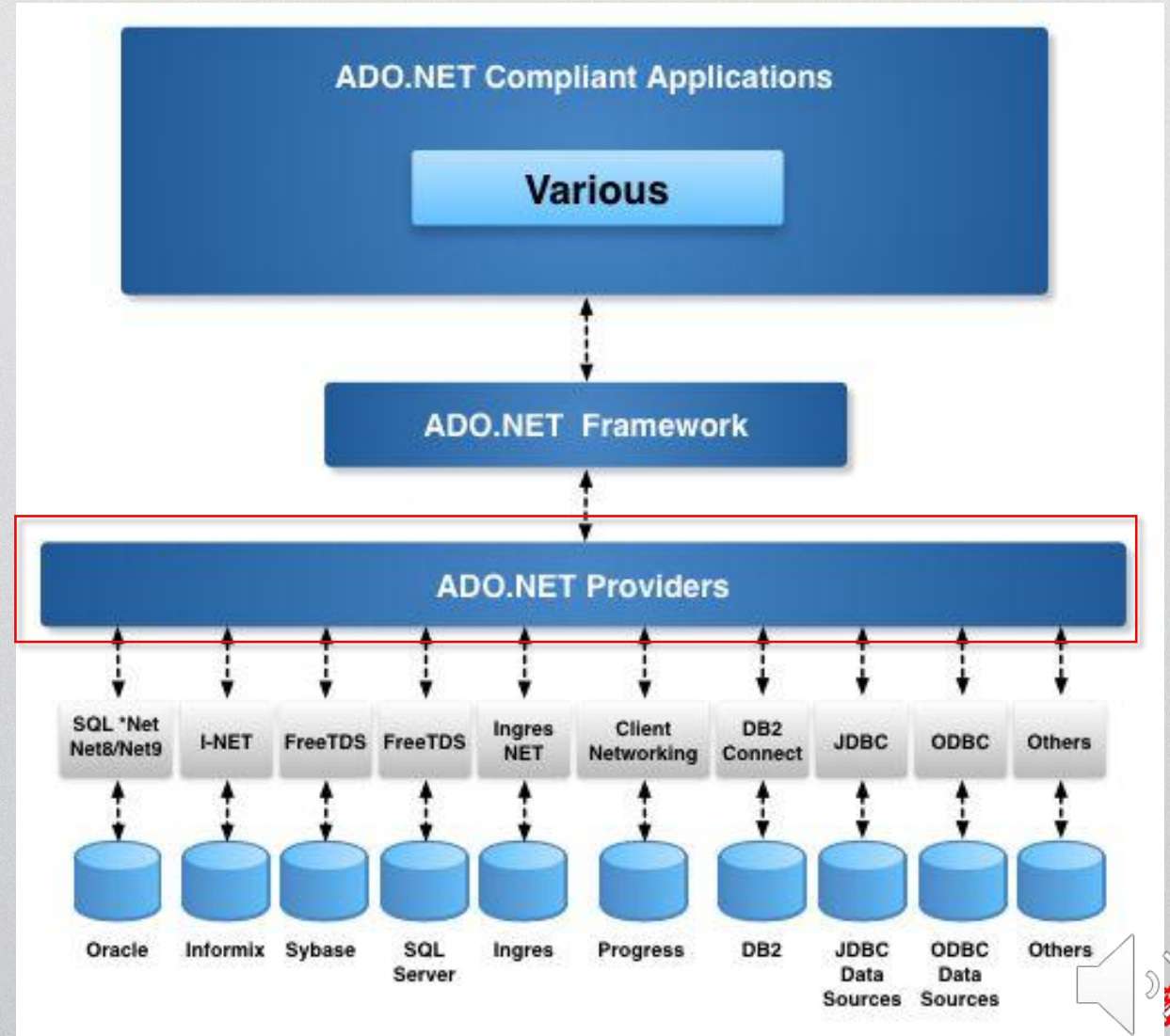




ADO.NET Data Provider

➤ Data Provider

- The class library used to access specific database
- Generally developed by the vendor of the database
- SQL provider (SqlServer)
 - System.Data.SqlClient
- MySql provider (MySql)
 - System.Data.MySqlClient
- ODBC provider (ODBC)
 - System.Data.Odbc
- Oracle provider (Oracle)
 - System.Data.OracleClient





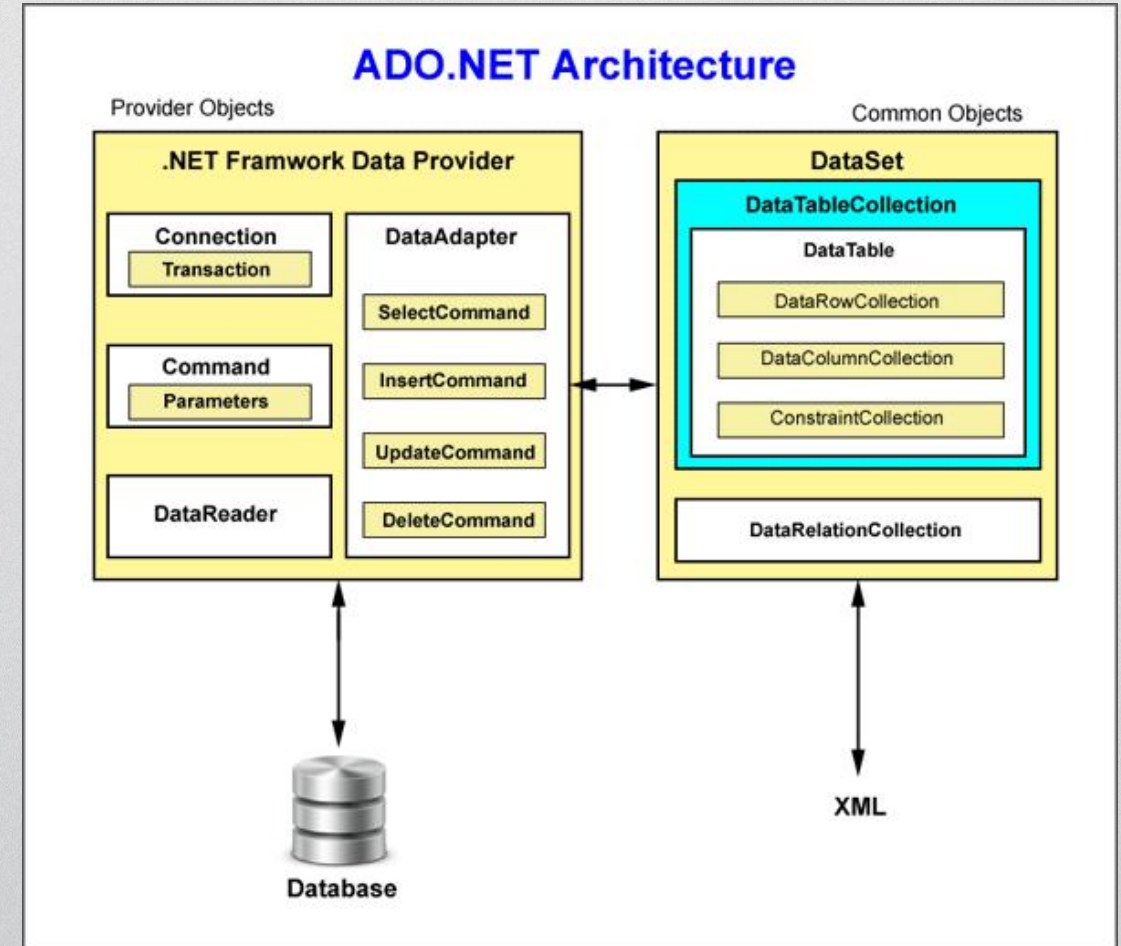
Architecture of ADO.Net

➤ Provider-specific classes

- **Connection
- **Command
- **DataAdapter
- **DataReader

➤ Common classes

- DataSet
- DataTable





Provider-specific classes

Classes	MySQL	SQL Server	Description
Connection	MySqlConnection	SqlConnection	建立与特定数据源的连接。
Command	MySqlCommand	SqlCommand	对数据源执行SQL命令 支持参数化命令
DataReader	MySqlDataReader	SqlDataReader	访问数据源的只读数据流。
DataAdapter	MySqlDataAdapter	SqlDataAdapter	数据库与DataSet之间的适配器。 用 DataSet 进行双向同步

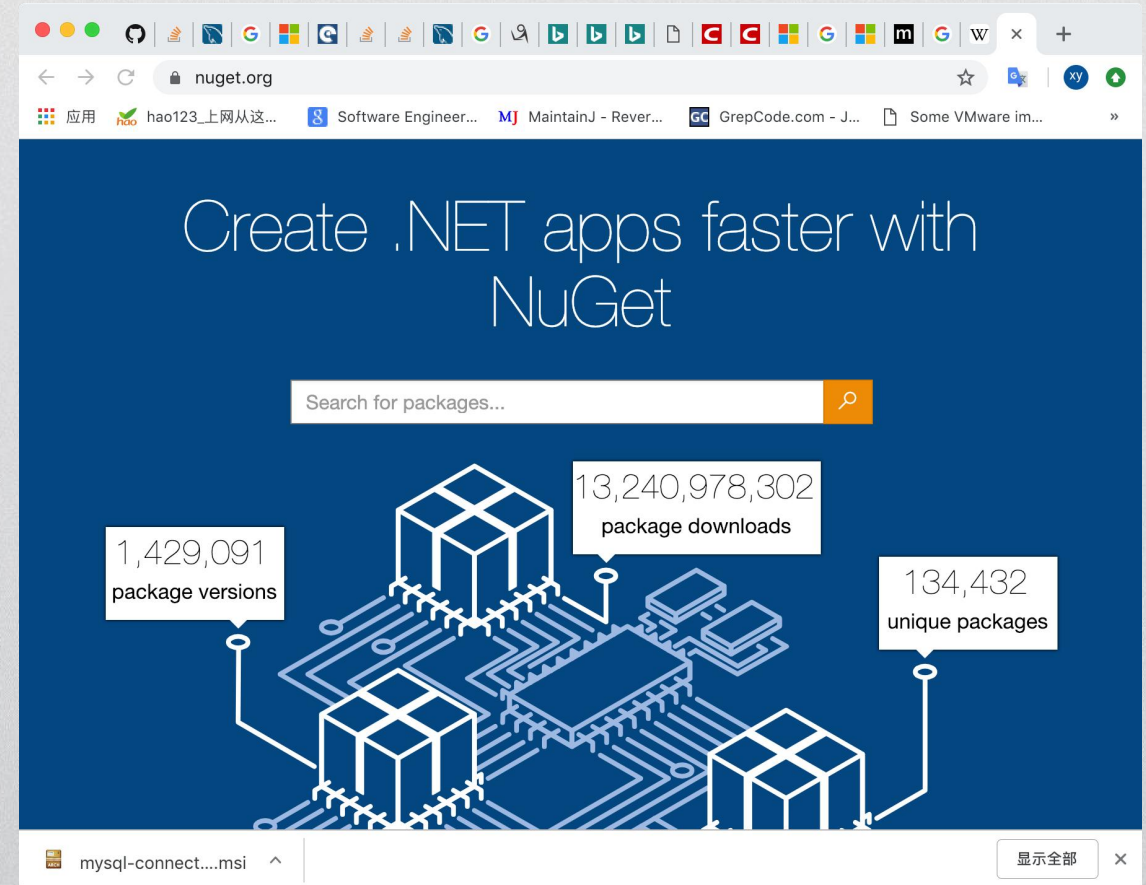


Nuget



Nuget

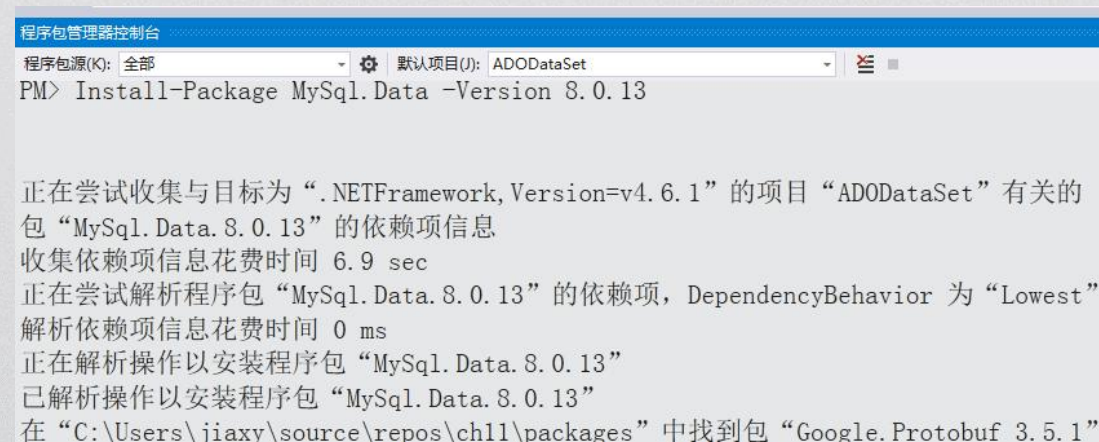
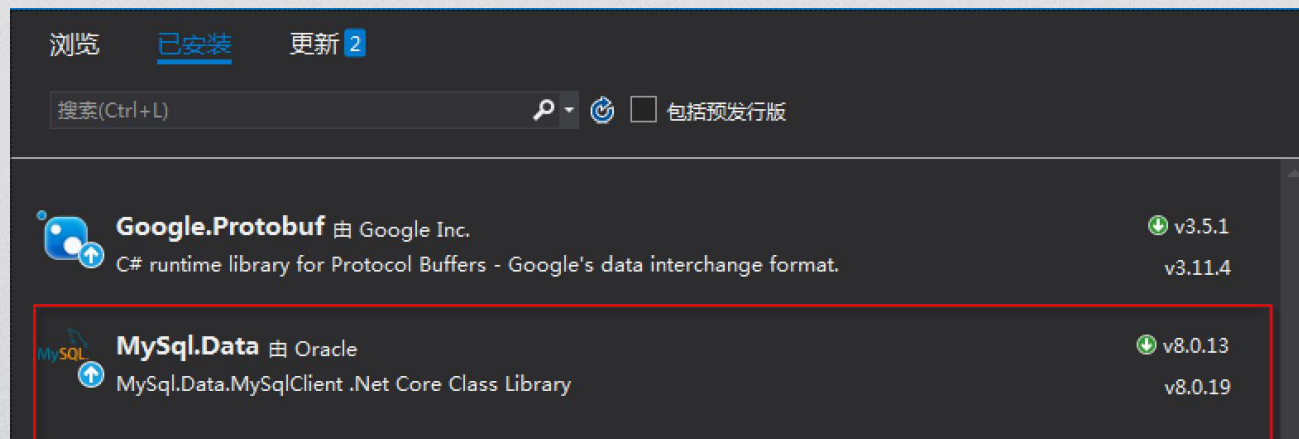
- Free and open source package management platform for Microsoft development
- Nuget is integrated in VS, for downloading and introducing third-party libraries into the project





Demo: Add MySQL provider into the project

➤ Nuget Package Manager, or Package Manager Console



注意：为避免兼容问题，MySQL.Data 的版本尽量和MySQL的版本保持一致。

Content

- Database Basics
- ADO.NET
- ADO Programming Basics
- Entity Framework





Database programming with ADO.Net

- **Solution1: programming with data provider**
 - Non-query operations: open the connection, execute the command, and close the link.
 - Query operations: open the connection, execute the command, get data with DataReader, and close the connection.
- **Solution2: programming with dataset (for single-use app)**
 - Open the connection, fill the DataSet with the DataAdaptor, read/write the data in the DataSet, save the DataSet, Close the connection.



• Programming with data provider

```
private static void DeleteAuthor() {  
    using ( MySqlConnection connection = GetConnection()) {  
        using ( MySqlCommand cmd = new MySqlCommand  
            ("delete from authors where Name= '莫言'", connection)) {  
            cmd.ExecuteNonQuery();  
        }  
    }  
}
```

Using{} automatically release connection and command resources

```
private static MySqlConnection GetConnection() {  
    MySqlConnection connection = new MySqlConnection(  
        "datasource=localhost;username=root;" +  
        "password=root;database=book;charset=utf8");  
    connection.Open();  
    return connection;  
}
```

Note: The connection strings are different for different database providers.





Execute a Command

- An command object executes commands upon the database after establishing a connection
 - `ExecuteNonQuery()`
 - Non-query commands (such as Update/Delete/Insert)
 - Returns the number of records it affects
 - `ExecuteReader()`
 - Execute a query command and return an Reader object
 - `ExecuteScalar()`
 - Get the result of a single value, such as sum, avg, etc.



Commend Parameters

```
cmd.CommandText = "INSERT INTO Nations(CName, EName, FName) VALUES  
(@CName, @EName, @FName);
```

```
cmd.Parameters.AddWithValue("@CName", "aaaaaa");
```

```
cmd.ExecuteNonQuery();
```

Parameters is easy-to-use and safer than string concatenation

(1) Special characters (" " %, etc.) in commands

```
cmd.CommandText = "Insert into Nataions(CName) Values(' " + name + " ' ) ";
```




Commend Parameters

(2) Prevent SQL injection attacks

```
cmd.CommandText = "SELECT * FROM users Where uname="+name+" AND  
password="+password;
```

Attacks: input following strings in the login page

```
name= "admin or 1=1" ; password= "admin or 1=1";
```

The actual SQL executed:

```
"SELECT * FROM users Where (uname=admin or 1=1) AND (password=admin or 1=1)" ;
```

```
// All user information is obtained
```





Commend Parameters

```
private static void InsertAuthor() {  
    using ( MySqlConnection connection = GetConnection()) {  
        using ( MySqlCommand cmd = new MySqlCommand  
            ("INSERT INTO Authors(Name) VALUES(@Name)", connection)) {  
            cmd.Parameters.AddWithValue("@Name", "莫言");  
            cmd.ExecuteNonQuery();  
        }  
    }  
}
```




DataReader

- DataReaders are object to query data in ADO.NET
- **SqlDataReader**
 - `SqlCommand.ExecuteReader()`: execute the comment and get a reader
 - `SqlDataReader.Read()`: read one record from the results
 - `SqlDataReader.Get***(int)` : get the specific column value of current record
`GetString(0), GetBoolean(1), GetInt16(2), GetInt32(3)....`



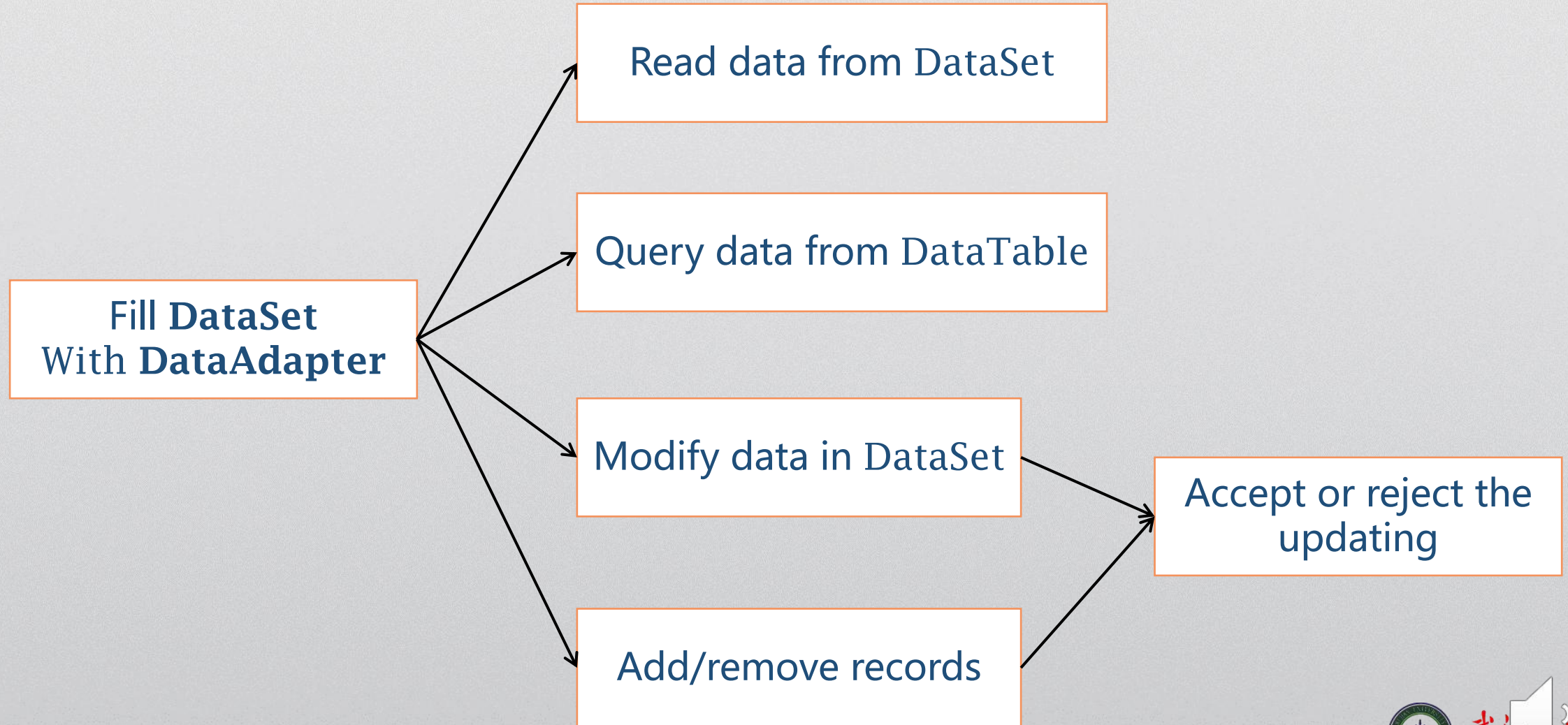
DataReader

```
private static void QueryAuthors() {  
    using (MySqlConnection conn = GetConnection()) {  
        string stm = "SELECT * FROM Authors";  
        using (MySqlCommand cmd = new MySqlCommand(stm, conn)) {  
            using (MySqlDataReader reader = cmd.ExecuteReader()) {  
                while (reader.Read()) {//逐行读取  
                    Console.WriteLine(reader.GetInt32(0) + ": "  
                        + reader.GetString(1));  
                }  
            }  
        }  
    }  
}
```





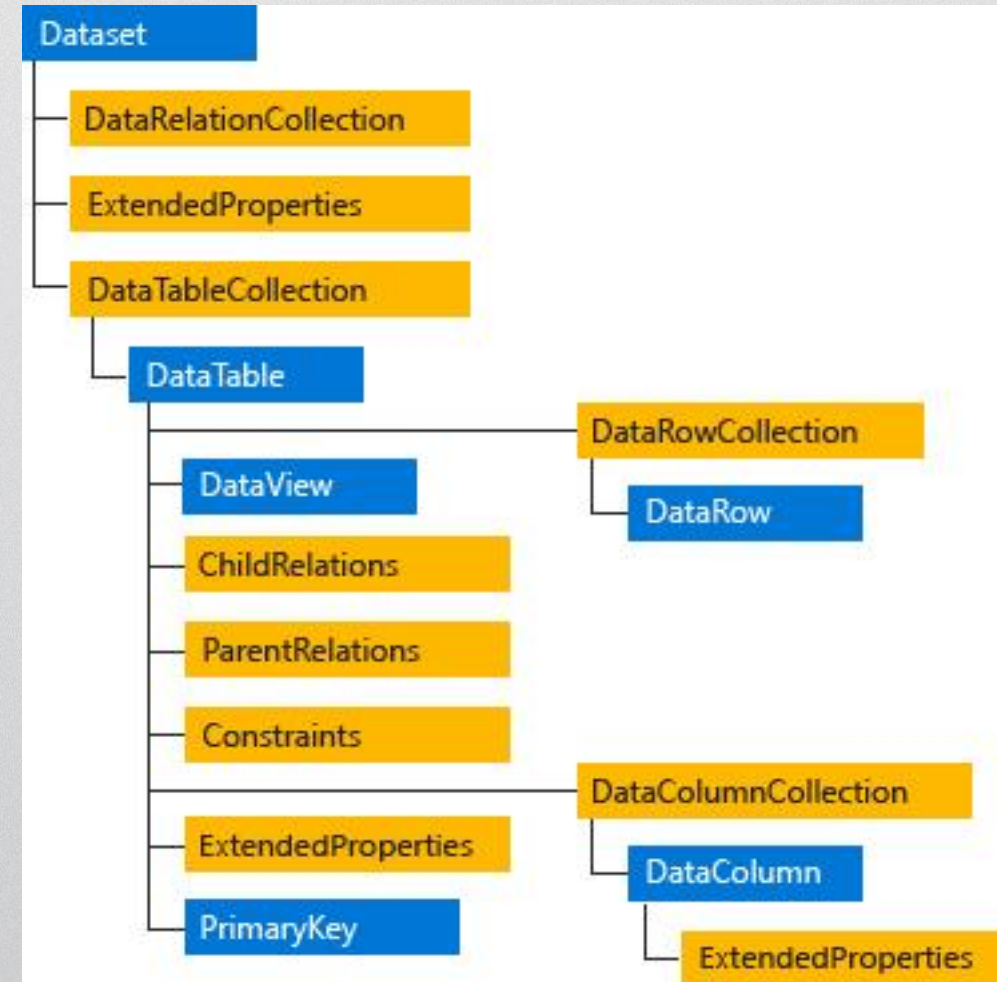
Solution2: Programming with DataSet





DataSet related classes

- DataSet
- DataTable
- DataRow
- DataColumn
- DataView
- Constraints
- ...





Demo: read data from DataSet

```
using (MySqlConnection conn = GetConnection()) {  
    String sql = "SELECT * FROM Authors";  
    using (MySqlDataAdapter dataAdapter = new MySqlDataAdapter(sql, conn)) {  
        using (DataSet ds = new DataSet()) {  
            dataAdapter.Fill(ds);  
            foreach (DataRow row in ds.Tables[0].Rows) {  
                foreach (object field in row.ItemArray) {  
                    Console.Write(field + "\t");  
                }  
                Console.WriteLine();  
            }  
        }  
    }  
}
```

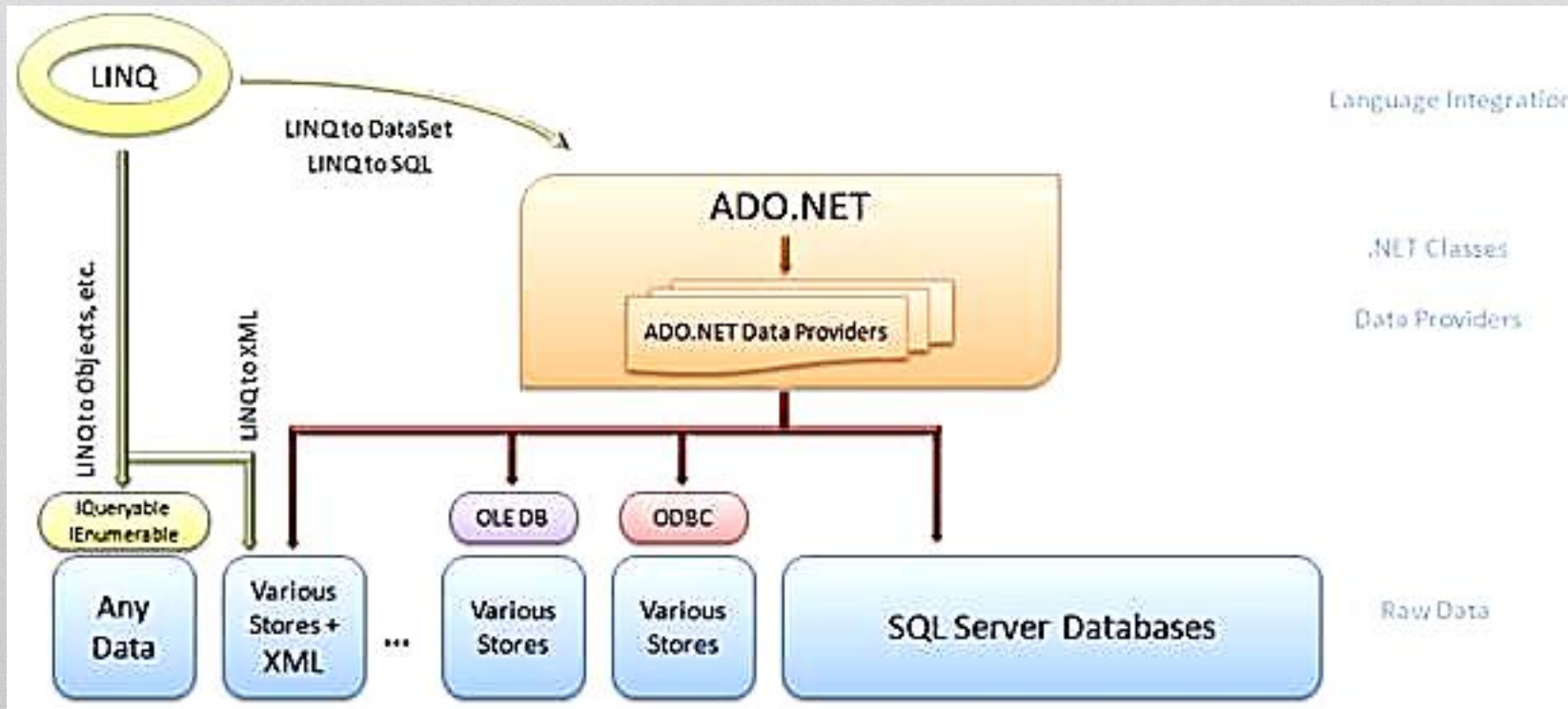
Read data using the row and column numbers :
Console.WriteLine(ds.Tables[0].Rows[0][1]);





LINQ to DataSet

```
DataRow[] rows = ds.Tables[0].Select("AuthorId=1");  
for (int i = 0; i < rows.Length; i++) {  
    ....  
}
```





Update data in DataSet

```
using (MySqlDataAdapter dataAdapter = new MySqlDataAdapter(sql, conn)) {  
    MySqlCommandBuilder cmdBuilder =  
        new MySqlCommandBuilder(dataAdapter);  
    using (DataSet ds = new DataSet()) {  
        dataAdapter.Fill(ds);  
        DataRow[] rows = ds.Tables[0].Select("AuthorId=1");  
        for (int i = 0; i < rows.Length; i++) {  
            rows[i].BeginEdit();  
            rows[i][1] = 6;  
            rows[i].EndEdit();  
        }  
        dataAdapter.Update(ds); //submit changes to database  
    }  
}
```





Add records to DataSet

```
DataRow newRow = ds.Tables[0].NewRow();  
newRow[1] = 6;  
newRow[2] = "New Book";  
ds.Tables[0].Rows.Add(newRow);  
....  
dataAdapter.Update(ds); //submit changes to database
```

Content

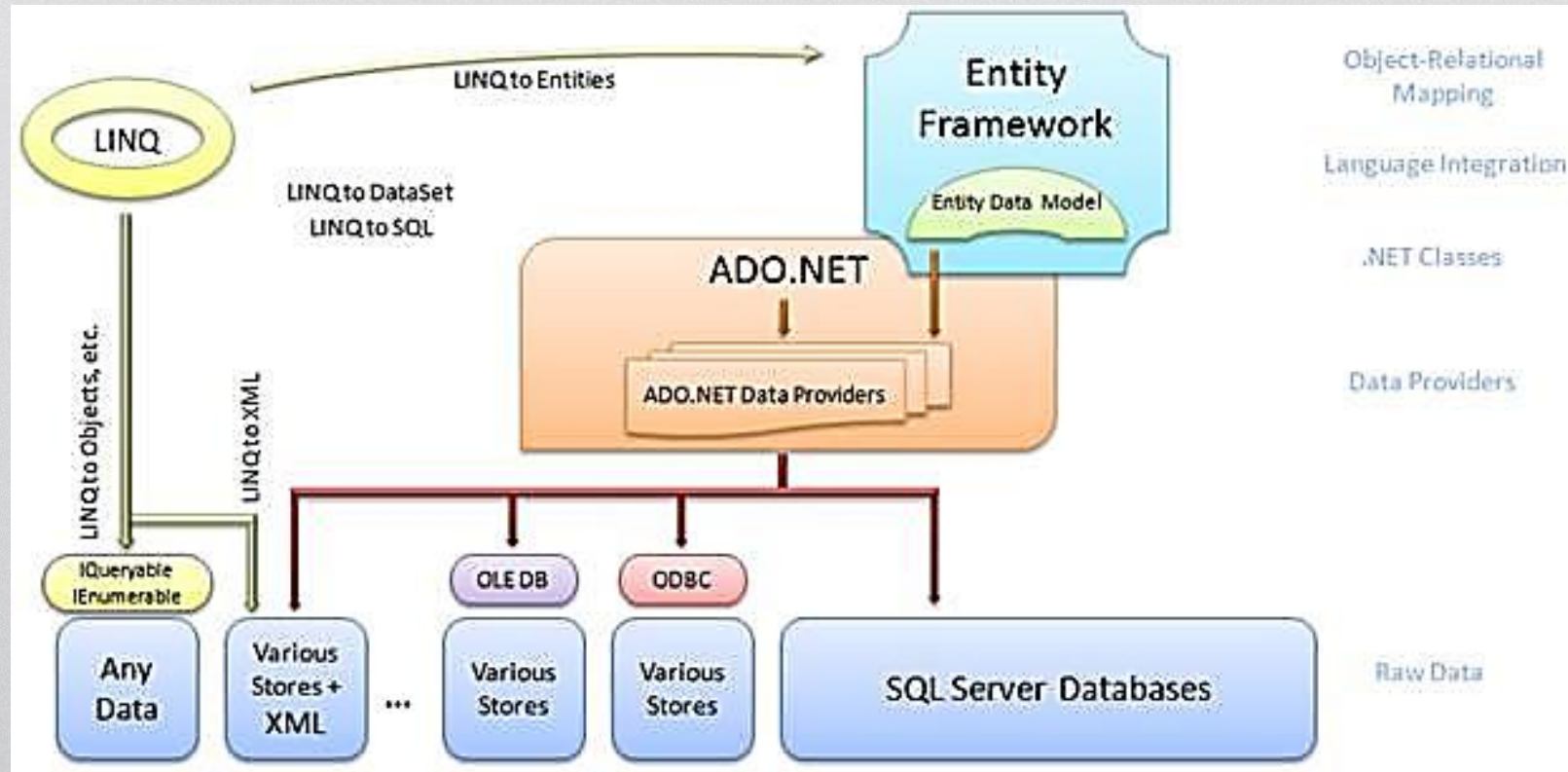
- Database Basics
- ADO.NET
- ADO Programming Basics
- Entity Framework





Entity Framework

- An ORM(object-relational mapping) framework based on ADO.Net





ORM (object-relational mapping)

■ ORM

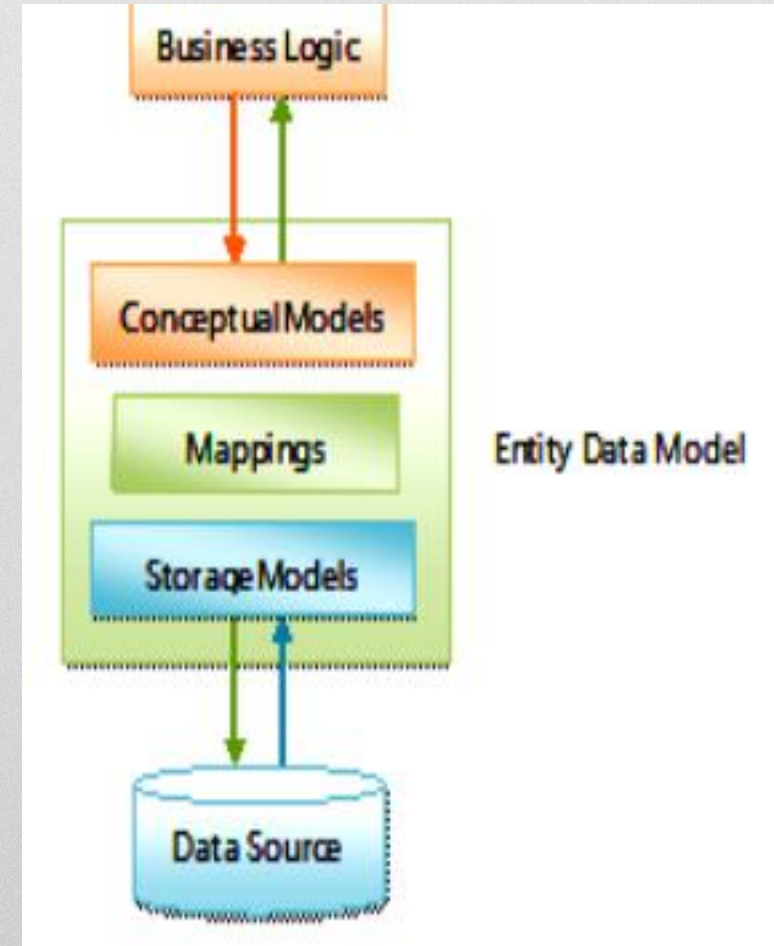
- ❑ Read/write the data by manipulating the object.

■ Why use ORM?

- ❑ Ease to use
- ❑ More object-oriented, and better type safety

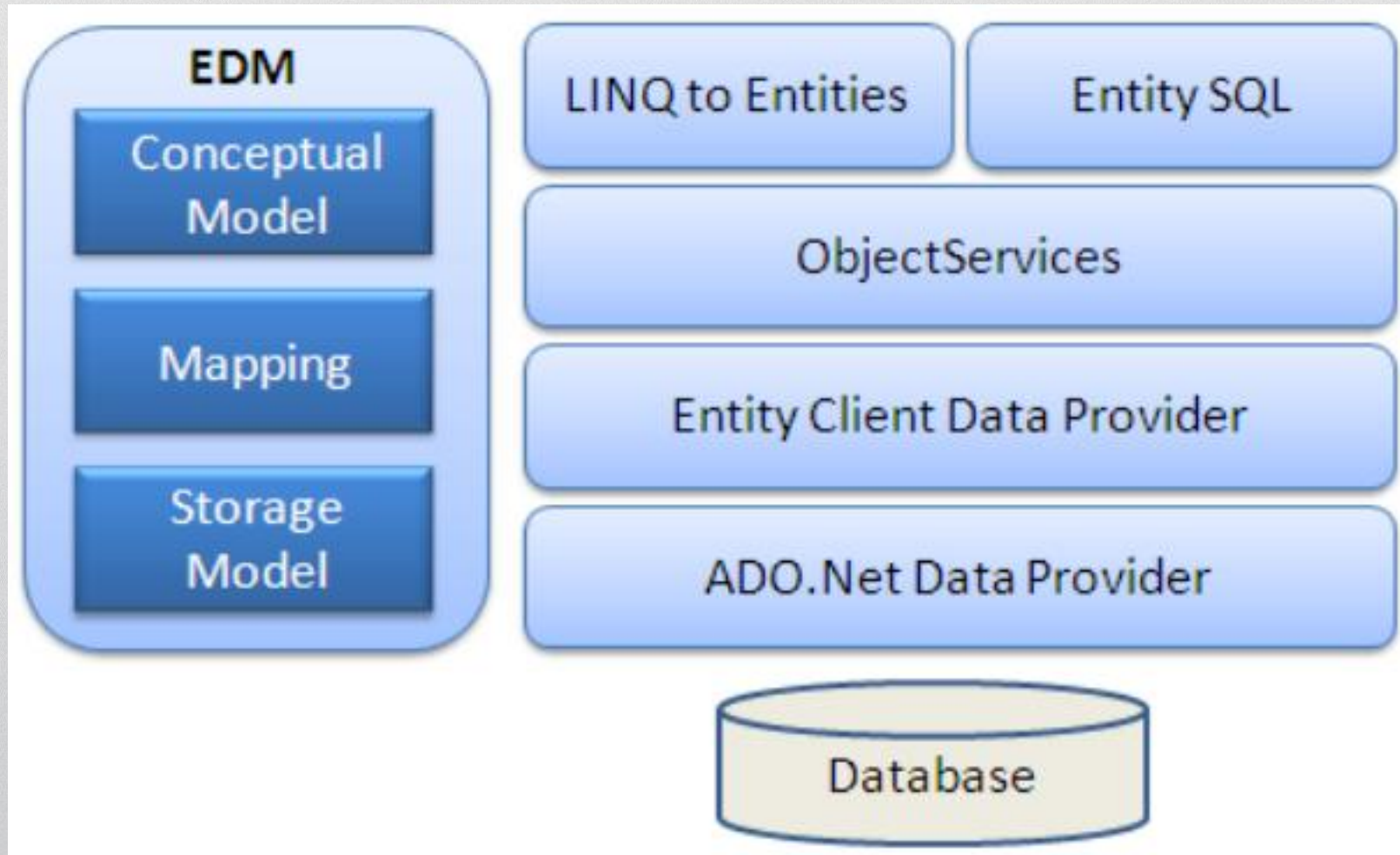
■ Popular ORM frameworks

- ❑ Java: Hibernate, JPA, MyBatis
- ❑ .Net: EF



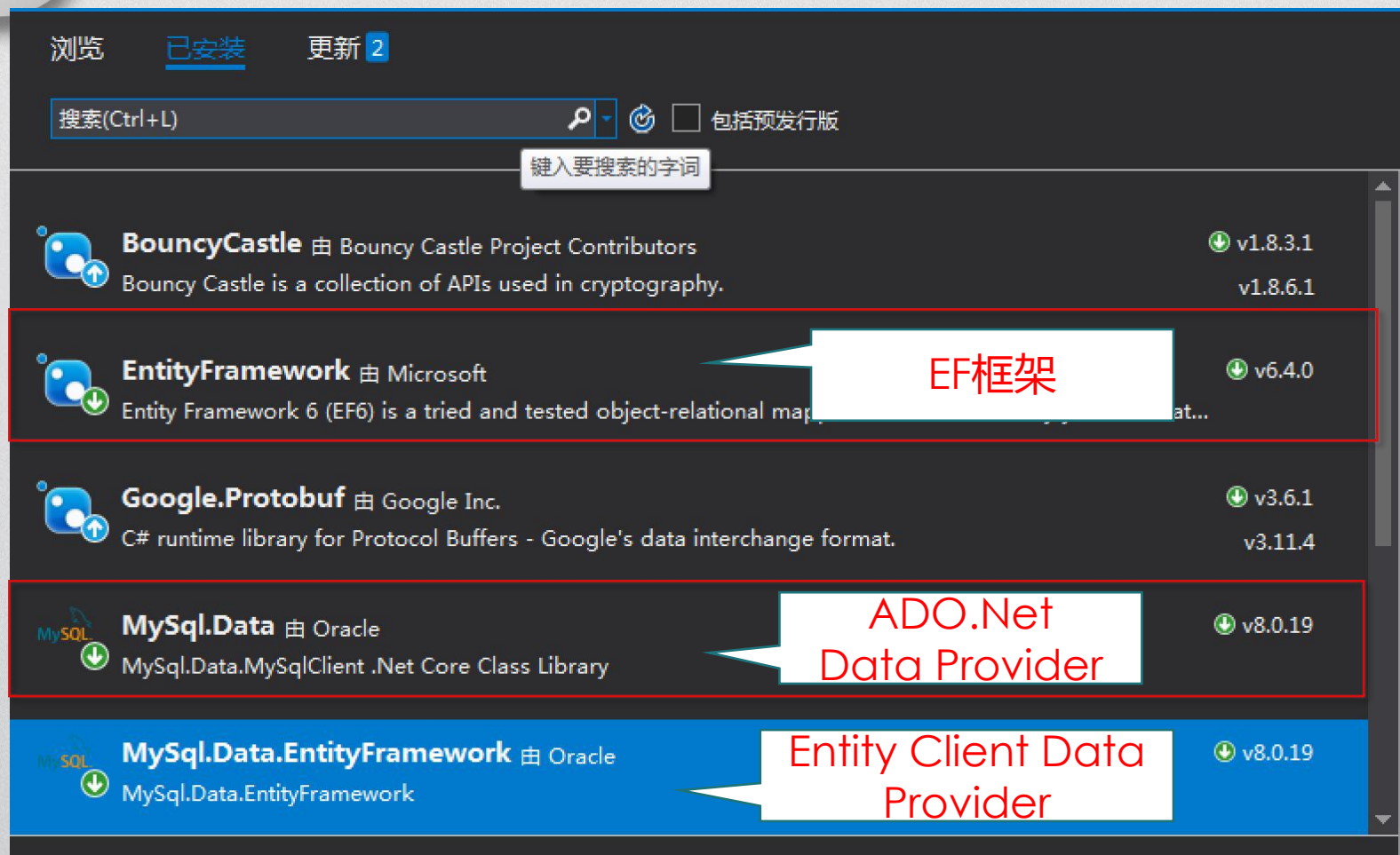


Architecture of EF(Entity Framework)





Add packages of EF



安装MySQL.Data.EntityFramework会自动安装对应版本的MySQL.Data
(另外, 本机需要预先安装 MySQL connector for .Net)



Usage of EF

- Code First : write code first, and generate database tables
- Model First: draw design first, generate database tables and code (requires VS support)
- Database First: generate code or design from database (requires VS support)





Demo: Code-First programming with EF

➤ Entity Classes and tables

Database and Tables are generated when the program is executed.

```
mysql> show tables;
+-----+
| Tables_in_blogdb |
+-----+
| __migrationhistory |
| blogs              |
| posts              |
+-----+
3 rows in set (0.00 sec)

mysql>
```

```
public class Blog {
    public int BlogId { get; set; } //自动识别为主键
    public string Url { get; set; }
    public int Rating { get; set; }
    public List<Post> Posts { get; set; } //一对多关联
}

public class Post {
    public int PostId { get; set; } //自动识别为主键
    public string Title { get; set; }
    public string Content { get; set; }
    public string Comment { get; set; }

    public int BlogId { get; set; } //自动识别为外键
    public Blog Blog { get; set; } //多对一关联
}
```




DbContext

```
public class BloggingContext : DbContext {  
    public BloggingContext() : base("BlogDataBase") {  
        Database.SetInitializer(  
            new DropCreateDatabaseIfModelChanges<BloggingContext>());  
    }  
  
    public DbSet<Blog> Blogs { get; set; }  
    public DbSet<Post> Posts { get; set; }  
}
```

ConnectionString
name

Database Name

<!-- 这部分是开发人员编写的，不同的数据库驱动连接字符串不同 -->

```
<connectionStrings>  
    <add name="BlogDataBase"  
        connectionString="server=localhost;user id=root;password=root;database=blogDB"  
        providerName="MySql.Data.MySqlClient" />  
</connectionStrings>
```

<!-- 下面的部分是安装Nuget的 EntityFramework和MySql.Data.EntityFramework之后自动生成的 -->

```
<entityFramework>  
    <providers>  
        <provider invariantName="System.Data.SqlClient" type="System.Data.SqlClient" />  
        <provider invariantName="MySql.Data.MySqlClient"  
            type="MySql.Data.MySqlClient.MySqlProviderServices" />  
    </providers>  
</entityFramework>
```

Data provider name





Add data into database

//添加博客

```
using (var db = new BloggingContext()) {  
    var blog = new Blog { Url = "http://sample.com", Rating = 3 };  
    blog.Posts = new List<Post>() {  
        new Post() { Title = "Test1", Content = "Hello"},  
        new Post() { Title = "Test2", Content = "Hello"}  
    };  
    db.Blogs.Add(blog);  
    db.SaveChanges();  
}
```

Add the associated objects together

Add into Blogs and saveChanges

Set the foreign key

//添加帖子

```
using (var db = new BloggingContext()) {  
    var post = new Post() { Title = "Test3", Content = "Hello", BlogId = 1 };  
    db.Entry(post).State = EntityState.Added;  
    db.SaveChanges();  
}
```

Modify the State of object in DbContext

Note: When adding records, exceptions will be thrown if the primary key is duplicated, the non-null field is empty, or if the foreign key value is invalid, etc.





States of entity object in DbContext

❑ Detached

- The object exists, but is not tracked by DbContext. The entity is in this state after it was created but before being added to the DbContext.

❑ Added

- The object is new and has been added to the DbContext, but not yet saved

❑ Deleted

- The object has been removed from the DbContext

❑ Modified

- Properties on the object has been changed, but not yet preserved

❑ Unchanged

- This object has not been modified since the object was attached to the dbcontext, or has not changed since the last SaveChange call



Delete Data from database

❑ Delete Data with EF

- Get the object from DbContext, and then delete it

```
//删除Id为1的帖子
using (var context = new BloggingContext()) {
    var post = context.Posts.FirstOrDefault(p => p.PostId == 1);
    if (post != null) {
        context.Posts.Remove(post);
        context.SaveChanges();
    }
}
```

```
//删除Id为1的博客及其帖子
using (var context = new BloggingContext()) {
    var blog = context.Blogs.Include("Posts").FirstOrDefault(p => p.BlogId == 1);
    if (blog != null) {
        context.Blogs.Remove(blog);
        context.SaveChanges();
    }
}
```

Note: When deleting record from the master table, an exception is thrown if the slave table has a foreign key pointing to the record.

"Include" checks out the corresponding records of the slave table.



Update data

❑ Update data with EF

- Solution1: create an object with same primary key, and set its state as Modified
- Solution2: query the object from DbContext, and modify it

```
//修改Id为1的帖子的Comment
using (var context = new BloggingContext()) {
    var post = new Post() { PostId = 1, Title = "Test3", Content = "Hello", BlogId = 1 };
    context.Entry(post).State = EntityState.Modified;
    post.Comment = "test! ";
    context.SaveChanges();
}
```

```
//修改Id为1的帖子的Comment
using (var context = new BloggingContext()) {
    var post = context.Posts.FirstOrDefault(p => p.PostId == 1);
    if (post != null) {
        post.Comment = "test! ";
        context.SaveChanges();
    }
}
```




LINQ to Entity

- Most syntax are the same with LINQ to Object
 - Select/Where/Order By

- Differences
 - The query type of LINQ to Entity is `IQueryable<T>` instead of `IEnumerable<T>`
 - When querying master-slave tables, the result of the slave table will not be returned automatically.



Basic query

//根据Id查找博客

```
using (var context = new BloggingContext()) {  
    var blog = context.Blogs  
        .SingleOrDefault(b => b.BlogId == 1);  
    if(blog!=null) Console.WriteLine(blog.Url);  
}
```

//查询评分大于1的博客

```
using (var db = new BloggingContext()) {  
    var query = db.Blogs  
        .Where(b => b.Rating > 1)  
        .OrderBy(b => b.Url);  
    foreach (var b in query) {  
        Console.WriteLine(b.BlogId);  
    }  
}
```




Query from multiple tables

- Query by field of the associated table

```
//查询url为"http://sample.com"的博客的所有帖子
using (var db = new BloggingContext()) {
    var query = db.Posts
        .where(p => p.Blog.Url == "http://sample.com")
        .OrderBy(p => p.Title);
    foreach (var p in query) {
        Console.WriteLine(p.Title);
    }
}
```

(Blog is a property of Post)

- Query the master table, and the result data includes related data from slave tables

```
//查询评分大于1的博客 (包括所有帖子)
using (var db = new BloggingContext()) {
    var query = db.Blogs.Include("Posts")
        .where(b => b.Rating > 1)
        .OrderBy(b => b.Url);
    foreach (var b in query) {
        Console.WriteLine(b.BlogId);
    }
}
```

- ❑ **Include("propertyName")**: fetching records from the master table will includes records from the slave tables
- ❑ Requirement: class Blog has a property named Posts



Query from multiple tables

- Multi-table query with “join”

```
var Result3 = from p in db.Masters
               join q in db.Details on p.Id equals q.MasterID
               select new {
                   Id = p.Id,                (Masters has no “Details” property)
                   m1 = p.MasterField1,
                   m2 = p.MasterField2,
                   d1 = q.DetailField1,
                   d2 = q.DetailField2};
```




Pagination

Skip/Take:

```
var users = db.Users
    .OrderBy(o => o.Name)
    .Skip(10)
    .Take(5); //从第11个记录开始读，最多取5个
```

分页的实现:

```
int currentPage=2; int pageSize=10;
var users = db.Users
    .OrderBy(o => o.Name)
    .Skip((currentPage-1)*pageSize)
    .Take(pageSize);
```




Attributes in EF

- **What does the EF framework create tables based on?**
- **“Convention over configuration” (约定优于配置)**
 - ❑ EF assume that your class follows the conventions of EF, and can get information automatically from your class to create the tables.
 - ❑ If your class does not adhere to these conventions, you need to add configuration to the class to provide EF with the information it needs.
- **Two ways to add configuration**
 - ❑ Data Annotations: add EF attributes to entity classes
 - ❑ Fluent API: add configuration with code



Basic Conventions

■ Primary Key

- ❑ The property named ID, or "className+ID"

■ Foreign key

- ❑ "ForeignClassName+ID"

■ Navigation Properties

- ❑ Collection properties for one-to-many relationship
- ❑ Non-collection properties for many-to-one, or one-to-one

```
public class Department
{
    // Primary key
    public int DepartmentID { get; set; }
    public string Name { get; set; }

    // Navigation property
    public virtual ICollection<Course> Courses { get; set; }
}

public class Course
{
    // Primary key
    public int CourseID { get; set; }

    public string Title { get; set; }
    public int Credits { get; set; }

    // Foreign key
    public int DepartmentID { get; set; }

    // Navigation properties
    public virtual Department Department { get; set; }
}
```

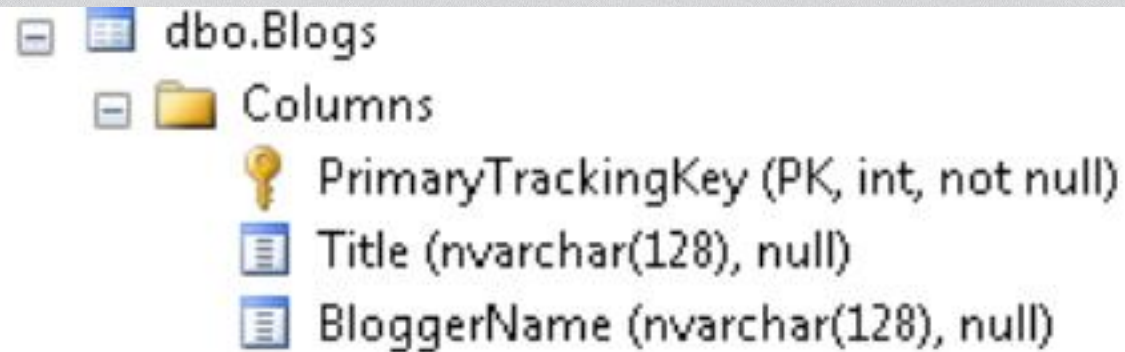




Key

- **[key]** indicates that this property is the primary key

```
public class Blog
{
    [Key]
    public int PrimaryTrackingKey { get; set; }
    public string Title { get; set; }
    public string BloggerName { get; set; }
    public virtual ICollection<Post> Posts { get; set; }
}
```





ForeignKey

[ForeignKey] tells EF to generate a foreign key in the table for the navigation property

```
public class Post
{
    public int Id { get; set; }
    public string Title { get; set; }
    public DateTime DateCreated { get; set; }
    public string Content { get; set; }
    public int BlogId { get; set; }
    [ForeignKey("BlogId")]
    public Blog Blog { get; set; }
    public ICollection<Comment> Comments { get; set; }
}
```




Required

- The field/property cannot be null

```
[Required]  
public string Title { get; set; }
```

```
[-] db.Blogs  
  [-] Columns  
    PrimaryTrackingKey (PK, int, not null)  
    Title (nvarchar(128), not null)  
    BloggerName (nvarchar(128), null)
```




MaxLength and MinLength

```
[MaxLength(10),MinLength(5)]  
public string BloggerName { get; set; }
```

MaxLength 注释将通过把属性长度设置为 10 来影响数据库。

[-]	[Folder]	Columns
[Key]		PrimaryTrackingKey (PK, int, not null)
[Table]		Title (nvarchar(128), not null)
[Table]		BloggerName (nvarchar(10), null)



Other attributes

- **NotMapped**: the property is not stored into the table
- **Table**: The configuration of the generated table
- **Column**: The configuration of the table column

```
[Table("InternalBlogs")]  
public class Blog
```

```
[Column("BlogDescription", TypeName="ntext")]  
public String Description {get;set;}
```




Assignment

- 修改之前做的订单程序，基于EF框架，将订单保存到MySQL数据库中，并实现订单的增删改查功能。